

SQL 조인과 서브쿼리 핵심 정리

오늘의 강의 학습 요약

금일 강의는 SELECT 실행 흐름(From→Where→Group By→Having→Select→Order By)을 다시 짚은 뒤, 함수 정리 관점(데이터 타입별 분류)과 그룹 함수/그룹핑 규칙을 재정리하고, 본격적으로 **조인(Join)** 문법과 동작을 체계적으로 다뤘습니다. 특히 이너 조인과 아우터 조인의 차이, ANSI 조인 문법(크로스/내추럴/USING/ON), 조인 조건과 검색 조건(WHERE)의 구분, 3개 이상 테이블 조인 확장 방식을 강조했습니다.

이어서 **서브쿼리(Subquery)** 개념을 조인과 비교해 정의하고, 서브쿼리의 실행 순서(서브쿼리→메인쿼리), 사용 위치(WHERE/FROM/SELECT 등), 단일행, 복수행 결과에 따른 연산자 선택(=, > vs IN/ANY/ALL/EXISTS)을 실습 예시로 정리했습니다. 추가로 FROM절 서브쿼리인 **인라인 뷰(Inline View)**를 성능 관점에서 소개했습니다.

1. SELECT 실행 순서와 함수 정리

SELECT 문은 FROM→WHERE→GROUP BY→HAVING→SELECT→ORDER BY 순서로 처리되며, 단일행 함수와 그룹 함수의 적용 단위가 다릅니다.

SELECT 는 테이블을 먼저 확정(FROM)한 뒤 조건으로 레코드를 필터링(WHERE)하고, 필요 시 그룹핑(GROUP BY) 및 그룹 조건(HAVING)을 적용한 다음, 최종 컬럼을 선택(SELECT)하여 정렬(ORDER BY)합니다. 이 처리 순서를 기준으로 쿼리 해석과 디버깅이 정리됩니다.

- FROM 절은 조회 대상(테이블 및 조인 결과)을 먼저 확정하는 단계입니다.
- WHERE 절은 행 단위로 레코드를 필터링하는 검색 조건입니다.
- GROUP BY 절은 행들을 그룹 단위로 묶어 집계를 가능하게 합니다.
- HAVING 절은 그룹 결과를 다시 필터링하는 조건입니다.

함수는 데이터 타입별로 정리하는 것이 유지에 유리합니다. 문자열/숫자/날짜/조건/형변환처럼 “데이터 종류에 따라 제공되는 함수군”으로 분류하면 API 문서를 따라가며 확장하기 쉽습니다. 특히 날짜 함수는 DBMS마다 문법 차이가 크고 동의어도 많으므로, “외우기”보다 “존재를 알고 필요 시 문서에서 찾는 방식”이 적합합니다.

문자열 길이 함수에서는 LENGTH 와 CHAR_LENGTH 차이를 주의해야 합니다. LENGTH 는 바이트 기준이라 한글(3바이트)처럼 멀티바이트 문자는 글자 수와 다르게 나올 수 있고, 보통 글자 수가 필요하면 CHAR_LENGTH 가 더 적절합니다.

단일행 함수는 “행마다 적용되어 기존 레코드 수만큼 결과가 나오는 함수”이고, 그룹 함수는 “그룹 단위로 합계/평균/최대/최소/개수 등을 계산하는 집계 함수”입니다. 그룹 함수는 GROUP BY로 명시적으로 그룹핑할 수도 있고, GROUP BY가 없으면 테이블 전체를 하나의 그룹으로 묵시적으로 처리합니다.

그룹 함수 사용 시 규칙이 중요합니다. SELECT 절에 그룹 함수와 일반 컬럼이 함께 오면 일반 컬럼은 반드시 GROUP BY에 포함되어야 합니다. 또한 COUNT는 COUNT(컬럼) 과 COUNT(*) 가 다르게 동작하며, 전자는 NULL을 제외하고, 후자는 전체 레코드 수를 계산합니다.

구분	핵심 의미	대표 예시	주의점
단일행 함수	행마다 적용되어 행 수 유지가 기본입니다.	문자열/날짜/형변환 함수 등	결과 행 수가 기본적으로 변하지 않습니다.
그룹 함수	그룹 단위로 집계되어 행 수가 감소합니다.	SUM, AVG, MIN, MAX, COUNT	일반 컬럼은 GROUP BY에 포함되어야 합니다.
COUNT(컬럼)	NULL을 제외하고 카운트합니다.	COUNT(comm)	NULL 처리 의도를 명확히 해야 합니다.
COUNT(*)	레코드 전체를 카운트합니다.	COUNT(*)	조인 결과의 레코드 수 변화에 유의해야 합니다.

핵심 키워드

- #FROM
- #WHERE
- #GROUP BY
- #HAVING
- #COUNT(*)

2. 조인 개념과 ANSI 조인 문법

조인은 여러 테이블에 분산된 데이터를 연결해 조회하는 기술이며, **이너 조인(일치만)**과 **아우터 조인(누락 포함)**이 핵심 분류입니다.

조인은 원하는 데이터가 하나의 테이블에만 있지 않고 여러 테이블에 분산되어 있을 때, 테이블 간 연결고리(PK-FK 등)를 통해 원하는 결과를 만들어내는 방식입니다. 조인은 일반적으로 **이너 조인**이 기본이며, 일치하지 않는 데이터는 **누락**될 수 있다는 점이 중요합니다.

조인은 큰 분류로 다음 관점들이 함께 등장합니다. (1) **이너/아우터(누락 포함 여부)**, (2) **ANSI 표준 문법 여부**(이 강의는 ANSI 조인 중심), (3) **연결 조건이 동등 비교인지/범위 비교인지**에 따른 **이퀴/논이퀴 조인**입니다.

분류 기준	종류	핵심 포인트	예시 상황
누락 처리	이너 조인	연결되는(일치하는) 레코드만 남습니다.	부서 40에 사원이 없으면 결과에서 부서 40이 사라 집니다.
누락 처리	아우터 조인	일치하지 않는 레코드까지 포함합니다.	모든 부서를 출력해야 하면 부서 40도 포함해야 합니다.
연결 방식	이퀴 조인	= 동등 비교로 연결합니다.	EMP.deptno = DEPT.deptno
연결 방식	논이퀴 조인	BETWEEN , < , > 등 범위 비교로 연결합니다.	EMP.sal BETWEEN SALGRADE.losal AND SALGRADE.hisal
문법 범위	ANSI 조인	DBMS 독립적으로 이식성이 좋습니다.	MySQL/Oracle 등에서 표준 문법으로 사용합니다.

조인 문법은 FROM 절에서 수행되며, 조인 이후에는 WHERE/GROUP BY/HAVING/ORDER BY 등을 그대로 이어 붙일 수 있습니다. 특히 “조인 조건”과 “검색 조건(WHERE)”을 분리해 이해해야, 복잡한 쿼리에서 조건 위치를 올바르게 배치할 수 있습니다.

- 조인 조건은 테이블을 연결하기 위한 조건이며 FROM 절의 JOIN 구문에 포함됩니다.
- 검색 조건은 조인 결과에서 추가 필터링하는 조건이며 WHERE 절에 작성됩니다.
- 조인 후에도 GROUP BY/HAVING/ORDER BY는 일반 SELECT와 동일하게 적용됩니다.
- 쿼리(Query)는 실무에서 SELECT와 동의어처럼 사용되는 경우가 많습니다.

핵심 키워드

#INNER JOIN

#OUTER JOIN

#ANSI JOIN

#조인 조건

#검색 조건

3. 이너 조인: CROSS, NATURAL, USING, ON

이너 조인은 기본 조인 형태이며, CROSS는 카티션 곱, NATURAL/USING은 공통 컬럼 기반, ON은 가장 범용적으로 조건을 명시합니다.

1) CROSS JOIN

CROSS JOIN은 연결 조건 없이 두 테이블의 모든 행을 서로 곱하는 방식(카티션 곱)입니다. 결과 행 수는 $T1 \text{ 행 수} \times T2 \text{ 행 수}$ 이며, 일반적으로 의미 있는 비즈니스 데이터 조회에는 부적합합니다.

예시로 EMP(14행)와 DEPT(4행)를 CROSS JOIN하면 56행이 출력됩니다. 부서번호가 맞지 않아도 무조건 결합되므로, "정상 조인처럼 보이지만 쓸모 없는 결과"가 될 수 있습니다.

```
SELECT *  
FROM emp  
CROSS JOIN dept;
```

항목	내용
결과 행 수	테이블1 행 수 × 테이블2 행 수입니다.
조인 조건	존재하지 않습니다.
실무 활용	의도적으로 모든 조합이 필요할 때만 제한적으로 사용됩니다.

2) NATURAL JOIN

NATURAL JOIN은 두 테이블의 "이름이 같은 공통 컬럼"을 자동으로 찾아 그 컬럼들로 동등 조인합니다. 공통 컬럼이 없으면 조인이 성립하지 않으며, 공통 컬럼이 여러 개면 "쌍으로 모두 일치"하는 조건으로 묶어 조인됩니다.

다만 공통 컬럼이 코드에서 명시되지 않으므로, 제3자가 쿼리를 볼 때 어떤 컬럼으로 조인했는지 즉시 파악하기 어렵고 가독성이 떨어질 수 있습니다. 또한 "컬럼명이 같아야만" 동작하므로, FK 컬럼명이 다르게 설계된 경우에는 사용할 수 없습니다.

```
SELECT *  
FROM emp  
NATURAL JOIN dept;
```

3) USING 절

USING은 NATURAL JOIN의 “가독성 문제”를 보완해, 공통 컬럼을 명시적으로 지정합니다. 지정한 컬럼은 양쪽 테이블에 모두 존재해야 하며, 여러 컬럼을 지정하면 모두 일치해야 조인됩니다.

중요한 제약으로, USING절 내부의 컬럼에는 테이블 별칭(예: `e.deptno`)을 붙일 수 없습니다. USING 자체가 공통 컬럼을 전제로 하기 때문에 컬럼명만 작성해야 합니다.

```
SELECT e.empno, e.ename, e.sal, d.dname
FROM emp e
JOIN dept d USING (deptno);
```

4) ON 절

ON은 조인 조건을 가장 명시적으로 표현하며, 컬럼명이 달라도 조인이 가능하고 논이퀴 조인(범위 조인)도 가능합니다. 또한 조인 조건에 연산자(=, BETWEEN 등)가 드러나므로 의도가 명확해 가독성이 높습니다.

단, ON 조인에서 양쪽에 모두 존재하는 컬럼(예: deptno)을 SELECT 절에 단독으로 쓰면 “어느 테이블 컬럼인지 모호”해 에러가 날 수 있으므로, 반드시 `e.deptno` 처럼 소속(테이블/별칭)을 명시해야 합니다.

```
SELECT e.ename, e.sal, d.dname, e.deptno
FROM emp e
JOIN dept d ON e.deptno = d.deptno
WHERE d.dname = 'SALES';
```

문법	조인 조건 지정	컬럼명 동일 필요	특징
CROSS JOIN	불가	무관	카티션 곱이라 결과 폭증 가능성이 큼.
NATURAL JOIN	자동	필요	공통 컬럼이 코드에 보이지 않아 가독성이 떨어질 수 있습니다.
USING	공통 컬럼 목록	필요	공통 컬럼을 명시해 NATURAL보다 가독성이 좋습니다.
ON	임의 조건식	불필요	가장 범용적이며 논이퀴 조인도 지원합니다.

핵심 키워드

#CROSS JOIN

#NATURAL JOIN

#USING

#ON

#논이퀴 조인

4. 다중 테이블 조인과 조합 전략

3개 이상의 테이블 조인은 “앞의 조인 결과를 가상 테이블(X)로 보고 다음 테이블과 다시 조인한다”는 관점으로 확장됩니다.

3개 조인은 결국 “(T1 JOIN T2)의 결과”를 하나의 가상 테이블(X)로 간주하고, X를 T3와 다시 조인하는 구조입니다. 이 규칙은 4개, 5개 테이블로도 동일하게 확장됩니다.

또한 실무에서는 조인 방식이 하나로 고정되지 않고, 테이블 특성에 따라 NATURAL/USING/ON을 섞어 쓰는 조합이 가능합니다. 예를 들어 EMP와 DEPT는 공통 컬럼(deptno)로 USING이 적합하고, SALGRADE는 범위 조건이 필요하므로 ON + BETWEEN이 적합합니다.

```
SELECT e.ename, e.sal, d.deptno, d.dname, s.grade
FROM emp e
JOIN dept d USING (deptno)
JOIN salgrade s ON e.sal BETWEEN s.losal AND s.hisal;
```

- 조인 대상이 늘어나면 조인 조건이 “각 연결 단계마다” 필요해집니다.
- 공통 컬럼 기반 조인(USING/NATURAL)과 범위 조인(ON)을 혼합할 수 있습니다.
- 조인은 모두 FROM 절 구성 요소이며, 이후 WHERE 등 절을 그대로 이어 붙일 수 있습니다.
- 조인 결과 레코드 수가 예상보다 급증하면 CROSS JOIN 또는 조건 누락을 의심해야 합니다.

핵심 키워드

#3개 조인

#가상 테이블

#JOIN 조합

#BETWEEN

#레코드 수

5. 아우터 조인: LEFT, RIGHT와 누락 데이터

아우터 조인은 일치하지 않는 레코드를 포함하며, LEFT는 왼쪽(T1) 보존, RIGHT는 오른쪽(T2) 보존을 의미합니다.

이너 조인은 일치하는 데이터만 남기므로, 예제에서 DEPT의 40번처럼 매칭되는 EMP가 없으면 누락됩니다. 아우터 조인은 이런 누락을 포함해 출력합니다.

MySQL 기준으로 강의에서 다룬 아우터 조인은 LEFT/RIGHT이며, “양쪽 모두를 완전히 보존”하는 FULL OUTER JOIN은 MySQL에는 없고 Oracle 등 일부 DBMS에 존재합니다. MySQL에서 양쪽 누락을 모두 포함하고 싶다면 LEFT 결과와 RIGHT 결과를 UNION으로 합치는 방식이 필요할 수 있습니다.

LEFT OUTER JOIN은 왼쪽 테이블(T1)의 모든 행을 보존하며, 매칭이 없으면 오른쪽 컬럼들이 NULL로 채워집니다. RIGHT OUTER JOIN은 오른쪽 테이블(T2)의 모든 행을 보존합니다.

조인	보존되는 쪽	의미	예시
LEFT OUTER JOIN	왼쪽(T1)	T1은 매칭 여부와 무관하게 모두 출력됩니다.	부서 미배정(EMP.deptno NULL) 사원도 출력됩니다.
RIGHT OUTER JOIN	오른쪽(T2)	T2는 매칭 여부와 무관하게 모두 출력됩니다.	사원이 없는 부서(40번)도 출력됩니다.

강의에서는 누락 상황을 명확히 만들기 위해 EMP에 deptno = NULL 인 신입사원(홍길동)을 추가하여, 이너 조인 시 한쪽(사원)도 누락될 수 있음을 확인했습니다. 이후 LEFT를 쓰면 홍길동이 포함되고, RIGHT를 쓰면 40번 부서가 포함됨을 비교했습니다.

또한 “양쪽을 동시에 모두 포함하는 아우터 조인”이 MySQL에 없다는 점을 강조했고, 대안으로 집합 연산자(UNION 등)를 언급했습니다.

핵심 키워드

#LEFT OUTER JOIN

#RIGHT OUTER JOIN

#NULL

#누락 데이터

#UNION

6. 셀프 조인과 집합 연산자 개요

셀프 조인은 한 테이블을 별칭으로 분리해 자기 자신과 조인하며, FULL OUTER 부재 같은 한계를 UNION 등 집합 연산으로 보완할 수 있습니다.

셀프 조인은 물리적으로는 하나의 테이블이지만, 논리적으로 두 개 이상의 역할(사원/관리자 등)로 분리해 조인하는 방식입니다. EMP 테이블의 MGR(관리자 사번)이 같은 EMP 테이블의 EMPNO(사번)를 참조하는 구조에서 자주 발생합니다.

핵심은 “테이블을 두 번 쓰는 것처럼 보이도록 별칭을 다르게 부여”하고, 사원.mgr = 관리자.empno로 연결하는 조건을 만드는 것입니다. 이 방식은 관리자, 관리자의 관리자 등으로 확장도 가능합니다.

```
SELECT e.ename AS 사원명, m.ename AS 관리자명
FROM emp e
JOIN emp m ON e.mgr = m.empno;
```

강의 중 “MySQL에서 양쪽 누락(예: 홍길동과 40부서)을 동시에 보고 싶다”는 질문에 대해, FULL OUTER JOIN이 없으므로 LEFT/RIGHT 결과를 UNION으로 합치는 접근을 안내했습니다. 또한 UNION/INTERSECT 등은 집합 연산자(set operator)로 분류된다는 점을 언급했습니다.

- 셀프 조인은 한 테이블을 별칭으로 분리해 서로 다른 역할로 조인합니다.
- 집합 연산자는 두 쿼리 결과를 합치거나(UNION) 공통만 남기는 방식으로 확장할 수 있습니다.
- UNION으로 합칠 때는 두 SELECT의 컬럼 구조가 호환되어야 합니다.
- “결과 행 수가 예상보다 과도”하면 조인 조건 누락 또는 CROSS JOIN을 의심해야 합니다.

핵심 키워드

#SELF JOIN

#MGR

#별칭

#UNION

#집합 연산자

7. 서브쿼리 기본: 정의, 실행순서, 사용 위치

서브쿼리는 한 번의 SELECT로 결과를 만들 수 없을 때 SELECT를 중첩해 해결하며, 괄호로 감싼 서브쿼리가 먼저 실행됩니다.

조인은 "여러 테이블에 분산된 데이터를 연결"하는 방식이고, 서브쿼리는 "원하는 값을 얻기 위해 SELECT가 여러 단계로 필요"한 경우에 사용하는 문법입니다. 예를 들어 "SMITH보다 급여가 많은 사원"은 (1) SMITH 급여 조회, (2) 그 급여보다 큰 사원 조회의 두 단계가 필요하며, 이를 하나의 SQL 문으로 합친 것이 서브쿼리입니다.

서브쿼리는 반드시 괄호로 감싸야 하며, 괄호 없는 바깥 SELECT를 메인 쿼리라고 합니다. 실행 순서는 서브쿼리가 먼저 실행되고, 그 결과가 메인 쿼리 조건/데이터로 치환되어 메인 쿼리가 실행됩니다.

서브쿼리는 WHERE에만 제한되지 않고 FROM(인라인 뷰), SELECT, HAVING, 그리고 INSERT/UPDATE/DELETE/CREATE 등 다양한 위치에 올 수 있습니다. 다만 실무에서 가장 흔한 형태는 WHERE 절의 서브쿼리입니다.

항목	내용
표기	서브쿼리는 반드시 (...) 괄호로 감쌉니다.
용어	괄호 없는 바깥 쿼리는 메인 쿼리, 괄호 쿼리는 서브쿼리입니다.
실행 순서	서브쿼리 실행 후 그 결과로 메인 쿼리가 실행됩니다.
위치	WHERE/FROM/SELECT/HAVING 및 DML/DDI 전반에 올 수 있습니다.

핵심 키워드

#서브쿼리

#메인쿼리

#실행 순서

#WHERE

#HAVING

8. 서브쿼리 연산자: 단일행 vs 복수행(IN/ANY/ALL/EXISTS)

서브쿼리 결과가 1행이면 단일행 연산자(=, > 등)를, 여러 행이면 복수행 연산자(IN, ANY, ALL, EXISTS)를 사용해야 합니다.

서브쿼리 결과가 1개 값(단일행)인 경우에는 =, >, < 같은 비교 연산자를 사용해도 됩니다. 강의에서 다른 평균 급여(AVG)나 특정 부서의 최소 급여(MIN)처럼 집계로 1개 값이 나오는 경우가 대표적입니다.

반면 “업무별 최소급여”처럼 그룹바이로 여러 값이 반환되면, = 같은 단일행 연산자를 쓰면 오류가 발생합니다. 이때는 IN 같은 복수행 연산자를 사용해야 합니다.

복수행 연산자 중 ANY/ALL은 “최솟값/최댓값” 패턴을 자연스럽게 표현할 때 자주 쓰입니다. ALL은 “모든 값과 비교해 모두 만족”해야 하고, ANY는 “값들 중 하나만 만족”하면 됩니다. 강의에서는 다음 대응을 중심으로 결과가 동일함을 확인했습니다.

요구사항 해석	다중행 연산자 표현	단일값으로 풀어쓴 동치
최솟값보다 작다	< ALL (서브쿼리)	< (SELECT MIN(...) ...)
최댓값보다 크다	> ALL (서브쿼리)	> (SELECT MAX(...) ...)
최솟값보다 크다	> ANY (서브쿼리)	> (SELECT MIN(...) ...)
최댓값보다 작다	< ANY (서브쿼리)	< (SELECT MAX(...) ...)

EXISTS는 서브쿼리 결과의 “값” 자체가 아니라 “존재 여부”를 기준으로 메인 쿼리를 실행할지 결정합니다. 서브쿼리 결과가 한 건이라도 존재하면 메인 쿼리가 실행되고, 존재하지 않으면 메인 쿼리는 실행되지 않아 결과가 비게 됩니다.

```
SELECT '값'
WHERE EXISTS (SELECT 1 FROM emp WHERE sal = 800);
```

핵심 키워드

#단일행 연산자

#복수행 연산자

#IN

#ANY/ALL

#EXISTS

9. 인라인 뷰(FROM절 서브쿼리)와 성능 관점

인라인 뷰는 FROM 절에 서브쿼리를 배치해 “미리 집계/필터링된 결과”를 테이블처럼 조인에 참여시켜 성능을 개선할 수 있습니다.

인라인 뷰는 FROM 절에 “테이블 대신 서브쿼리”를 두는 형태입니다. 서브쿼리 결과는 논리적으로 테이블처럼 취급되며, 별칭을 반드시 부여해 이후 조인 대상으로 사용할 수 있습니다.

강의 예시는 “부서별 총합과 평균” 같은 집계 결과를 먼저 만들고, 그 결과를 DEPT와 조인하는 방식이었습니다. 이렇게 하면 조인에 참여하는 레코드 수가 줄어들어(예: EMP 15행 전체가 아니라 부서별 집계 4행) 대용량에서 성능 이점이 생길 수 있습니다.

```
SELECT v.deptno, d.dname, v.sum_sal, v.avg_sal
FROM (
  SELECT deptno, SUM(sal) AS sum_sal, AVG(sal) AS avg_sal
  FROM emp
  GROUP BY deptno
) v
JOIN dept d ON v.deptno = d.deptno;
```

비교 포인트	일반 조인 후 집계	인라인 뷰로 사전 집계
조인 참여 행 수	원본 테이블 행이 크게 참여합니다.	집계 후 결과 행만 참여합니다.
성능 경향	대용량에서 비용이 커질 수 있습니다.	대용량에서 비용을 줄일 여지가 큼니다.
형태	조인→GROUP BY 순으로 작성되는 경우가 많습니다.	FROM에 서브쿼리(뷰)를 두고 조인합니다.

핵심 키워드 #인라인 뷰 #FROM 서브쿼리 #성능 #사전 집계 #별칭